

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВА-
НИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет прикладной математики, информатики и механики
Кафедра вычислительной математики и информационных
прикладных технологий
Направление 01.04.02 Прикладная математика и информатика

Отчёт

по производственной практике (научно-исследовательская работе)

Тема	Разработка аппаратной реализации 2D гра- фического ускорителя
Отчетный семестр	4

Обучающийся	2 курс, 11 группа, Черкасов В.В.
Руководитель от кафедры	к.ф.-м.н., доц. Медведева О.А.

Воронеж 2026

Содержание

Введение	3
1. Постановка задачи	5
2. Актуальность разработки	6
3. Стратегия верификации блока	8
3.1. Тип тестирования	8
3.2. Метод тестирования	10
3.3. Почему Mesa 3D?	12
3.4. Выбор драйвера	12
4. Используемые технологии	15
5. Реализация тестового окружения	16
5.1. Устройство драйвера	16
5.2. Модификация драйвера	16
5.3. Работа с симуляцией СФ-блока	17
6. Бекенд компилятора с промежуточным представлением NIR	18
6.1. Промежуточное представление NIR	18
6.2. Архитектура системы команд вычислительного ядра	20
6.3. Парсер	22
6.4. Транспилятор	23
6.5. Оптимизатор	24
6.6. Генератор кода	25
6.7. Ассемблер и линковщик	25
Заключение	26
Литература	27

Введение

Современная компьютерная графика давно превратилась из задачи статической визуализации в одну из самых ресурсоёмких вычислительных дисциплин. Рендеринг одного кадра современной сцены требует обработки миллионов геометрических примитивов, выборки гигабайтов текстурных данных и вычисления миллиардов операций в шейдерах. Несмотря на стабильный рост производительности универсальных CPU, программная реализация графических пайплайнов на процессорах общего назначения быстро достигает архитектурного потолка. Причина кроется не в тактовой частоте, а в фундаментальном расхождении парадигм выполнения вычислений.

CPU оптимизирован для минимизации латентности последовательных операций, сложной предсказуемости ветвлений и максимальной универсальности. Графические же вычисления по своей природе массово-параллельны, обладают высокой пространственной и временной локальностью доступа к памяти, а также минимальным количеством условных переходов на уровне примитивов. Аппаратный графический ускоритель устраняет этот разрыв на уровне кремния: специализированные SIMD/SIMT-ячейки, аппаратные планировщики волн (warps/wavefronts), локальные кэши текстур и кадровых буферов, а также оптимизированные передачи данных позволяют выполнять тысячи независимых операций за один такт, минуя накладные расходы на декодирование инструкций, динамическую диспетчеризацию, общие системные шины и избыточную логику управления прерываниями.

Верификация RTL-описания[1] - это не этап валидации кода перед синтезом, а фундаментальный инженерный процесс, определяющий жизнеспособность всего проекта интегральной съемы. На уровне RTL архитектура кристалла уже зафиксирована в виде детерминированных логических

уравнений, конечных автоматов, конвейеров и структур данных. Ошибка, допущенная на этом этапе, перестает быть «багом в коде» и превращается в физический дефект кремния, устранение которого часто невозможно без перепроектирования и объявления всех уже изготовленных схем бракованными.

Целью этой работы является создать тестовое окружение для верификации работы вычислительного ядра. Для достижения поставленной цели необходимо выполнить следующие задачи:

1. Выбрать метод тестирования
2. Имплементировать выбранный метод тестирования

1. Постановка задачи

Компанией АО «ПКК» Миландр была поставлена задача разработать аппаратную реализацию 2D графического ускорителя. Перед началом аппаратной разработки необходимо описать архитектуру, которой должен соответствовать сложно-функциональный (СФ) блок.

При разработке архитектуры учитываются проделанные ранее обзор и анализ 2D графических ускорителей[2]. Разработка ведется с учетом предложенной ранее[3] архитектурой графического ускорителя.

Для разработки архитектуры необходимо учитывать контекст, в котором будет использоваться аппаратный блок. Основной целью разработки графического ускорителя (GPU) является снижение нагрузки на центральный процессор (CPU). Этого можно добиться путём вычисления сложных матричных операций на GPU, чтобы CPU мог заниматься более приоритетными задачами. Это сделает систему в целом более быстрой и отзывчивой.

2. Актуальность разработки

Несмотря на узкую специализацию применения аппаратного блока 2D графического ускорителя, подобные решения используются практически повсеместно в современных встраиваемых системах и устройствах с графическим пользовательским интерфейсом. При условии наличия дисплея высокого разрешения в любом современном устройстве — от промышленного контроллера до медицинского оборудования — требуется аппаратное ускорение отрисовки интерфейса для обеспечения плавной анимации, отзывчивости и минимизации нагрузки на центральный процессор.

Ввиду того, что изделия решают разные задачи в различных предметных областях, каждое из них предъявляет специфические требования к GPU, которые определяются сложностью визуализации, требованиями к энергопотреблению, надёжности и стоимости конечного продукта. Разработка универсального графического ускорителя, способного адаптироваться под широкий спектр применений, является сложной архитектурной задачей, требующей глубокого понимания областей эксплуатации и рабочих нагрузок.

Ниже представлена таблица, содержащая несколько примеров того, какие именно могут быть требования к GPU, в зависимости от устройства, которое будет его использовать:

Таблица 1 — Требования к GPU

Требование	Пример устройства
Минимальное энергопотребление и низкая стоимость	Умные часы, термостат, простая панель управления
Высокая надёжность, работа с несколькими слоями изображения и плавность анимации	Автомобильный экран (приборная панель, медиацентр), кассовый аппарат, промышленный контроллер

Требование	Пример устройства
Высокая производительность интерфейсов и продвинутые эффекты	Бюджетный смартфон, планшет

Дополнительно к ускорению отрисовки графики GPU нашло свое применение в качестве аппаратного ускорителя для нейронных сетей. Искусственный интеллект на основе нейронных сетей характеризуется высоким потенциалом к параллелизации вычислений и низким количеством ветвления. Это ставит массово-параллельные архитектуры в преимущественное положение относительно других. Будет справедливо заметить, что для наилучшего ускорения расчета нейронных сетей требуются специализированные инструкции, однако архитектура самого ускорителя остается полностью аналогичной.

Разработка собственного графического ускорителя и программного обеспечения к нему является стратегически важной задачей для обеспечения технологического суверенитета Российской Федерации в области микроэлектроники и программных систем. В текущих геополитических условиях, характеризующихся ограничениями на поставки западных технологий и компонентов, независимость в сфере графических вычислений приобретает особое значение.

3. Стратегия верификации блока

3.1. Тип тестирования

Конечным результатом работы разработчика RTL-описания является создание логического сложно-функционального блока (логический СФ-блок). В этой статье логический СФ-блок - это структурный элемент микросхемы, состоящий из моделей на языке описания аппаратуры, представленный в виде описания на синтезируемом подмножестве языка описания аппаратуры SystemVerilog[4].

После завершения процесса верификации такой блок будет изготовлен в кремне. Это является причиной основного отличия подхода к тестированию СФ-блоков от тестирования программного обеспечения (ПО). Блок, выпущенный в кремне, не может быть исправлен в уже изготовленных схемах, а потому в зависимости от типа ошибки может быть только объявлен полностью не работоспособным или обладающим ограниченной функциональностью. Это значительно повышает требования на процесс тестирования и верификации блока и время, затраченное на верификацию, может составлять до 80% от времени на весь проект[5].

Верификацию СФ-блока можно разделить на 5 типов[5]:

1. Функциональная верификация
2. Формальная верификация
3. Статический анализ кода
4. Физическая верификация
5. Прототипирование

Функциональная верификация направлена на проверку правильности работы системы в штатных и нештатных ситуациях, производится проверка соответствию блока техническому заданию. Это тестирование часто счита-

ется наиболее значимым, так как если блок не работает, то никому не интересно, насколько хорошо он не выполняет задачи.

Формальная верификация заключается в установлении эквивалентности моделей в процессе проектирования блока. В большей степени этот шаг выполняется компьютером и от человека требуется анализ полученных артефактов[5].

Статический анализ кода - полностью автоматический процесс, который указывает на места, в которых потенциально была допущена ошибка, методом поиска шаблонных ошибок.

Инструменты для формальной верификации и статического анализа кода разработаны и предоставлены компанией АО «ПМК Миландр», а потому не будут рассматриваться в этой работе.

Физическая верификация и прототипирование предполагают окончание функциональной верификации, и присутствие финализированного, если и не готового, логического СФ-блока. Так как, в рамки этой работы входит именно создание первой итерации синтезируемого RTL-описания для вычислительного ядра графического ускорителя (GPU), эти шаги излишни.

К первой итерации блока вычислительного ядра накладывается три требования: синтезируемость, корректность работы и расширяемость архитектуры. Она будет основой для анализа и сравнения предлагаемых оптимизаций в будущем. Это позволит объективно оценивать изменения и делать обоснованные решения о их имплементации.

В связи с выше приведенными рассуждениями в качестве проводимого типа тестирования была выбрана функциональная верификация. В результате нее СФ-блок вычислительного ядра будет проверен на корректность работы и его способность решать поставленную перед ним задачу.

Задачей вычислительного ядра в графическом ускорителе (GPU) является исполнение шейдера в несколько потоков одновременно.

3.2. Метод тестирования

Методы функционального тестирования можно разделить на два типа по содержанию:

- блочное тестирование
- интеграционное тестирование

Во время блочного тестирования проверяется соответствие блока предоставленной документации на него, доступность и корректная работа заявленного функционала. В результате такого тестирования будут обнаружены ошибки в блоке или документации. Основной проблемой такого тестирования можно выделить, что блок не будет проверен на его пригодность к решению задачи.

Интеграционное тестирование позволяет встроить блок в уже работающую систему и оценить его способность решать реальные задачи. Однако, это тестирование не позволяет полностью покрыть все возможности блока и потому недостаточно для заявления о полной работоспособности блока.

Приведенные требования к первой итерации блока однозначно унавливают метод тестирования. Тестирование должно проводиться в виде интеграции блока в систему для оценки его пригодности, так как документация уже будет продуктом разработки самого блока. В результате блочного тестирования блока могут возникать проблемы с его взаимодействием с окружающими его системами. Могут упускаться или оставаться в слепой зоне соглашения, которые и разработчиком, и верификатором принимаются как истинные, но которые в результате попытки встроить блок вычислительного ядра в его окружение потребуют частичной или полной смены архитектуры.

Имплементацией выбранного метода верификации было выбрано заменить части реализации драйвера в Mesa 3D[6], ответственные за выполнение шейдеров на управление симуляцией блока вычислительного ядра. Другими словами, сделать драйвер тестовым окружением. В результате такой замены блок окажется в условиях, приближенных к реальным в видеокарте. Блоку будет снаружи выставлена вычислительная задача, через интерфейс системной памяти предоставлены данные для вычисления и выгрузки результата работы, блоку также придется сигнализировать о возникающих ошибках и завершении исполнения.

В качестве еще одного преимущества такого решения является наличие уже готового референса вычислений. Так как, изначально драйвер Mesa 3D уже выполнял вычисление шейдера, можно сравнивать референсный результат вычислений и получившийся в результате симуляции. А в связи с тем, что это все-таки драйвер, а не конечное приложение, можно запускать такую проверку для разного рода приложений.

Однако, блок вычислительного ядра будет в первую очередь проверяться в симуляции, а не на выделенном железе, а потому сравнивать производительность программной реализации и СФ-блока не имеет смысла. Заранее утверждается, что в результате замены производительность упадет. Это вызвано необходимостью симулировать на процессоре работу цифрового блока, который выполняет указанные вычисления, вместо выполнения их напрямую. Конечный результат работы - синтезируемое RTL-описание цифрового блока вычислительного ядра - будет работать быстрее после его синтеза и реализации «в железе».

Для того, чтобы встроить симуляцию блока в драйвер, необходимо ответить на следующие вопросы:

1. Где и каким образом происходит исполнение шейдеров в драйвере?

2. Как передать данные из этапа компиляции шейдеров в промежуточное представление в этап его исполнения?
3. Где и каким образом в памяти хранятся объекты UBO (Universal Buffer Object)?
4. Где и каким образом в памяти хранятся вводные данные для шейдеров на этапе исполнения?
5. Куда и каким образом сохраняются результаты выполнения шейдеров?

3.3. Почему Mesa 3D?

Графическая библиотека Mesa 3D - это имплементация с открытым исходным кодом различных графических API стандартов: OpenGL для десктоп и встроенных систем, Vulkan, EGL,- API для вычислений общего назначения на GPU (GPGPU): OpenCL, - API аппаратных ускорителей видео: VA-API.

К моменту написания работы в Mesa 3D имплементированы и поддерживают открытые драйверы для видеокарт AMD, NVidia, Qualcomm, Broadcom, Vivante, открытые драйверы для виртуальных графических карт, используемых в VMWare и VirGL, полностью программные реализации драйверов, которые эмулируют работу видеокарты на процессоре[6].

Разработка программного обеспечения к произведенному блоку столь же важная задача, что и разработка самого блока. Без сопутствующего ПО СФ-блок не больше чем простая игрушка. Разработка на основе уже использующегося драйвера позволяет получить понимание и опыт разработчикам, а также позволяет автоматически поддерживать все выше перечисленные стандарты.

3.4. Выбор драйвера

Для препарирования был выбран драйвер lavapipe(llvmpipe)[7] в связи с тем, что:

1. Драйвер `lavapipe` - это полностью программная реализация на процессоре, где вычисление шейдеров выполняется в однопоточной среде.
2. Четко выделены этапы вычисления шейдеров, как отдельные вызовы JIT-компилятора для исполнения скомпилированного байткода.
3. Используется промежуточное представление NIR[8] вместо TGSI[9].

Полностью программная реализация драйвера упрощает модификацию драйвера в тестовое окружение, так как не требует изучения дополнительных особенностей в передаче данных только на этапе вычисления шейдеров. А однопоточная среда позволяет симулировать только один экземпляр блока. Полноценная симуляция RTL описания, в зависимости от размера блока, может накладывать значительные требования по памяти и ресурсам, замедляя работу драйвера.

Разделение вычисления шейдеров от остального кода, реализующего другие части видеокарты является практически требованием. Если такое разделение отсутствует, то процесс их подмены потребует высокое количество усилий. В драйвере `lavapipe` в функции `llvm_pipeline_generic` представлены вызовы шейдеров вершин, тесселяции и геометрии, а в функции `llvmpipe_fs_variant_linear_llvm` кодируется оставшийся шейдер фрагментов. Это дает конкретные места для вставки кода, управляющего симуляцией вычислительного ядра, не задевая остальные части виртуальной видеокарты.

Код шейдеров, поставляемый вместе с приложением в формате `SpirV` в соответствии со стандартом `Vulkan`[10], в `Mesa 3D` транпилируется в трехадресный код[11]. Трехадресный код - это вид промежуточного представления, который используется оптимизирующими компиляторами. Он позволяет реализовать архитектурно-независимые оптимизации, так как не содержит в себе информацию о количестве и размере регистров и доступных на архитектуре операциях. В компиляторах промежуточное

представление потом дособируется в ассемблер, специфичный для конкретного устройства.

Трехадресный код в Mesa 3D представлен двумя эквивалентными промежуточными представлениями: TGSI и NIR. Более старое промежуточное представление TGSI по сравнению с более новым NIR содержит коды для сложных операций, например инструкцию для расчета коэффициентов освещения. Эти сложные операции содержат контекст исполнения, что необходимо для оптимизаций. В рамки этой работы не входит написание качественного оптимизирующего компилятора в связи с ограничением по времени, поэтому контекстуальная информация в TGSI излишна. Намного проще обрабатывать промежуточное представление NIR, так как оно содержит простые коды операций, которые можно реализовать в качестве поддерживаемых операции ядра, и в Mesa 3D уже написана реализация оптимизирующего компилятора для перевода байткода SpirV в NIR.

Одно из промежуточных представлений Mesa 3D было выбрано вместо байткода SpirV в связи с тем, что SpirV является стандартом Vulkan, а Mesa 3D поддерживает несколько API взаимодействия с графикой, одним из которых является Vulkan[6]. Таким образом, поддержка промежуточного представления Mesa 3D позволяет автоматически поддерживать остальные графические API помимо Vulkan (например OpenGL) без необходимости дополнительных действий.

4. **Использованные технологии**

Использованные технологии сгруппированы по главам:

1. Реализация тестового окружения:
 1. Язык программирования C
 2. Язык программирования C++
 3. Язык описания аппаратуры SystemVerilog
 4. Система сборки meson, ninja
 5. Система сборки Makefile
 6. Симулятор описания аппаратуры verilator
 7. Профайлер callgrind
2. Бекенд компилятора с промежуточным представлением NIR:
 1. Язык программирования Python
 2. Библиотека-генератор парсеров и токенизаторов Lark

5. Реализация тестового окружения

5.1. Устройство драйвера

5.2. Модификация драйвера

Тестовое окружение реализовано на основе драйвера `lavapipe`, поэтому для него необходимо ответить на ранее поставленные вопросы.

Исполнение шейдеров представлено в виде вызова JIT-компилятора для шейдеров вершин, тесселяции и геометрии в функции `llvm_pipeline_generic`. Эти вызовы функции подменяются на код, управляющий симуляцией. Этап исполнения шейдера фрагментов закодирован с оставшейся частью обработки, и его невозможно отделить от остальной части, поэтому он останется как есть.

В функцию `llvm_pipeline_generic` передается уже LLVM-байткод, в который закодированы циклы для каждой вершины и некоторые встроенные инструкции, которые на вычислительном ядре не должны обрабатываться. Также LLVM-байткод есть результат докомпиляции промежуточного представления NIR под конечную архитектуру машины, а потому не подходит под требования работы. Поэтому на этапе докомпиляции кода NIR в байткод виртуальной машины LLVM было добавлено сохранение шейдера в трехадресном представлении NIR для передачи на этап исполнения через структуру `draw_vertex_shader`.

Функция `llvm_pipeline_generic` вызывается по одному разу для каждого `draw` запроса. В качестве одного из параметров в нее передается буффер памяти вершин, который содержит UBO объекты, и сразу же за ними располагаются входные параметры вершин. Структура `draw_context` содержит непрерывный участок памяти, в котором расположены данные, передаваемые на исполнение виртуальному вычислительному ядру, а также информация о том, какие буферы были запрошены, их размер и их рас-

положение в памяти. Вводные данные шейдеров вычисляются на основе номеров вершин, для которых была вызвана обработка.

Данные вычислений сохраняются в структуру `draw_vertex_info`, которая содержит информацию о количестве вершин, количестве атрибутов на вершину, а также значения вычисленных атрибутов друг за другом.

Этой информации достаточно для того, чтобы правильно встроить симуляцию вычислительного ядра и заменить часть драйвера с программной реализации вычислений на симулируемые, но аппаратные вычисления.

5.3. Работа с симуляцией СФ-блока

6. Бекенд компилятора с промежуточным представлением NIR

Для запуска кода шейдера на ядре необходимо дособрать промежуточное представление NIR (New Intermediate Representation) в машинный код, который можно будет загрузить в системную память.

Ручное преобразование NIR в машинный код возможно, но требует значительных усилий и внимательности: только шейдер вершин в vkcube содержит 133 инструкции. В связи с этим в проекте использовалась полуавтоматическая сборка NIR с использованием скрипта на языке программирования Python.

Бекенд компилятора состоит из нескольких стадий:

1. Парсер - загружает тело функции, написанное на промежуточном представлении NIR в память программы.
2. Транспилятор - преобразует промежуточное представление NIR в трех-адресный код с инструкциями архитектуры системы команд вычислительного ядра.
3. Оптимизатор - выполняет архитектурно-зависимые оптимизации.
4. Генератор кода - генерирует код программы на языке ассемблера.
5. Ассемблер и линковщик - завершают сборку программы в машинный код.

Эта организация соответствует рекомендованной структуре бекенда компилятора[12]. В этой работе считается, что на вход подается уже полностью оптимизированный код NIR.

6.1. Промежуточное представление NIR

Промежуточное представление NIR (New Intermediate Representation) [8] - одно из двух промежуточных представлений, которые использует Mesa 3D, второе это TGSI[9]. Оно представляет из себя трех-адресный код[11] с

достаточно простыми инструкциями, которые разделяются на следующие виды по смыслу выполняемой операции:

1. `nir_instr_type_deref` - рассчитывает положение данных в системной памяти и возвращает указатель
2. `nir_instr_type_load_const` - загрузка константного значения
3. `nir_instr_type_intrinsic` - объявляет, что необходимо выполнить аппаратно-зависимую операцию, логика выполнения которой зависит от архитектуры вычислительного ядра
4. `nir_instr_type_alu` - математические и логические операции
5. `nir_instr_type_call` - вызов функции, подготовка к ее вызову и возвращение из вызванной функции
6. `nir_instr_type_jump` - операции управление потоком исполнения для реализации локальных прыжков в ветвлении и циклах

В рассматриваемом примере (`vkcube`) используется 12 уникальных инструкций NIR:

1. `deref_var` - поиск расположения значения атрибута в системной памяти, регистрах или каком-либо другом механизме
2. `load_deref` - загрузка значения этого атрибута
3. `load_const` - загрузка константы: скаляр или вектор
4. `load_const_buf_base_addr_lvp` - вычисление адреса начала настроенного буфера по его индексу
5. `load_ubo` - загрузить значение UBO (Universal Buffer Object) из буфера
6. `fmul` - умножение чисел с плавающей точкой
7. `fmax` - максимальное из двух чисел с плавающей точкой
8. `fadd` - сложение чисел с плавающей точкой
9. `fdiv` - деление чисел с плавающей точкой
10. `fneg` - операции инверсии знака числа с плавающей точкой
11. `fsqrt` - взятие квадратного корня числа с плавающей точкой

12. store_deref - выгрузка значения этого атрибута

6.2. Архитектура системы команд вычислительного ядра

Архитектура системы команд (ISA) вычислительного ядра основана на наборе команд открытого стандарта RISC-V[13], и следует архитектуре регистр-регистр (load-store architecture). В такой архитектуре инструкции выполняют только операции, в которых операнды, результат и статус исполнения представлены регистрами, для работы с системной памятью предоставляются инструкции load/store, которые позволяют загрузить/выгрузить значение регистра в память. В противопоставление архитектуре регистр-регистр существуют архитектуры регистр-память и память-память.

В архитектуре вычислительного ядра на одну нить исполнения предусмотрено 30 векторных (уникален на нить) регистров общего назначения, которые могут быть перезаписываться и читаться всеми инструкциями по усмотрению разработчика. Также предоставлены 2 регистра специального назначения:

- регистр zero(x0), который всегда содержит значение 0 и не может быть перезаписан;
- регистр v_id(vertex id), который содержит индекс вершины для обработки.

В этой работе для записи инструкций используется синтаксис Intel[14], то есть регистр, в который будут загружены результаты инструкции, указан первым (сразу после буквенной мнемоники операции), например lw rd, 0(rs), что по смыслу в языке программирования C есть операция разыменования с подобным псевдокодом: $rd = *rs$ (тут rd и rs это регистры).

Условные обозначения:

- M (memory) - системная память

- rd (register destination) - регистр, в который будет загружен результат инструкции
- rsn (register source n, где n=1,2) - регистр, содержащий значение операнда
- imm (immediate) - значение, которое хранится как часть инструкции

Для разработки бекенда компилятора для промежуточного представления NIR были предоставлены инструкции ниже. В левой части представлена мнемоника ассемблера, в правой части псевдокод выполняемой операции.

Синтаксис	Мнемоника	Формат	Псевдокод
Работа с памятью			
lw rd, imm(rs1)	load word	L	$rd = M[rs1 + imm]$
sw rs1, imm(rs2)	store word	L	$rd = M[rs1 + imm]$
Арифметические и логические операции			
lui rd,imm	U	load upper immediate	$rd = imm \ll 20$
addi rd,rs1,imm	add immediate	L	$rd = rs1 + imm$
add rd,rs1,rs2	add	L	$rd = rs1 + rs2$
mul rd,rs1,rs2	multiply	L	$rd = rs1 * rs2$
fdiv rd,rs1,rs2	floating-point divide	F	$rd = rs1 / rs2$
fneg rd,rs1	floating-point negate	F	$rd = -rs1$
fsqrt rd,rs1	floating-point square root	F	$rd = \sqrt{rs1}$

Синтаксис	Мнемоника	Формат	Псевдокод
fmax rd, rs1	floating-point maximum	F	rd = max(rs1, rs2)
fadd rd,rs1,rs2	floating-point add	F	rd = rs1 + rs2
fmul rd,rs1,rs2	floating-point multiply	F	rd = rs1 * rs2
Управление потоком			
ret	return	L	return

Для упрощения работы с ассемблером также был добавлен макрос:

- li rd, imm (load immediate), который разбивается на комбинацию инструкций lui и addi.

6.3. Парсер

Бекенд компилятора написан на языке программирования Python для удобства прототипирования и возможности быстро изменять устройство компилятора вслед за изменяющейся архитектурой. Это позволяет сократить потраченное время на отдельную итерацию дизайна, но добавляет дополнительные трудности работы с представлением инструкций в памяти.

В Mesa 3D код шейдера представлен в виде набора структур на языке программирования C. Дебаг режим позволяет вывести представление NIR в виде текста, который можно загрузить и обработать без потери информации о необходимых для выполнения операциях. Для выполнения этой задачи используется парсер, который способен загрузить тело функции.

Парсер относится к классу LALR(1) парсеров и был сгенерирован с помощью библиотеки генератора парсеров Lark. Исходная грамматика парсера представлена в приложении В.

В качестве результата работы парсера выводится массив элементов класса `Instruction`, который содержит информацию о возвращаемом значении, коде операции, размере возвращаемого значения в словах и аргументах. Аргумент может быть представлен номером, литералом и/или константой-именем, значение которой будет указано позже.

6.4. Транспилятор

Полученный на выходе парсера массив инструкций представляет собой исходный код на промежуточном представлении NIR и не содержит никакой информации о внутреннем устройстве вычислительного ядра. Также считается, что исходный код был полностью оптимизирован до начала работы бекенда компилятора, поэтому необходимо преобразовать исходный код на семантически идентичную последовательность инструкций согласно архитектуры системы команд вычислительного ядра.

На этой стадии генерируется трех-адресное представление конечного кода, и не совершается никаких оптимизаций или смены порядка инструкций для лучшего соответствия устройству вычислительного ядра.

Псевдокод работы транспилятора:

1. Для каждой инструкции исходного кода:
 1. Найти соответствие аргументов номеров исходного кода с конечным кодом
 2. Поставить соответствие команды в конечном коде
 3. Зарегистрировать соответствие номер результата конечного кода с исходным кодом

В следующей таблице представлено соответствие, которое использует транспилятор. Для простоты считается, что все параметры представлены в виде скаляров. Для векторных значений можно найти точный код соответствия в листинге В.

Исходная инструкция	Конечный код	Соответствие результатов
%0 = deref_var &name	li t0, name_base li t1, name_stride mul t2, v_id, t1 add t3, t0, t2	%0 = t3
%1 = @load_deref (%0)	lw t1, 0(t0)	%1 = t1
%0 = load_const (NNNN)	li t0, NNNN	%0 = t0
%1 = @load_const_buf_base_addr_lvp (%0)	li t0, ubo_base	%1 = t0
%2 = @load_ubo (%0, %1)	add t2, t0, t1 lw t3, 0(t2)	%2 = t3
%2 = fmul %0, %1	fmul t2, t0, t1	%2 = t2
%2 = fadd %0, %1	fadd t2, t0, t1	%2 = t2
%2 = fmax %0, %1	fmax t2, t0, t1	%2 = t2
%2 = fdiv %0, %1	fdiv t2, t0, t1	%2 = t2
%1 = fneg %0	fneg t1, t0	%1 = t1
%1 = fsqrt %0	fsqrt t1, t0	%1 = t1
%2 = store_deref %0, (%1)	sw t0, 0(t1)	—

6.5. Оптимизатор

Перед генерацией кода стоит сделать хотя бы простые архитектурно-зависимые оптимизации:

1. Удаление повторных загрузок констант
2. Удаление загрузки константного значения 0 и добавление ссылок на регистр zero
3. Удаление одинаковых инструкций
4. Удаление висящих инструкций

В качестве допущения к исходному коду NIR было принято, что код уже представлен в наиболее оптимизированной форме, однако это не так. Несколько инструкций загружают одни и те же значения констант в разные ячейки памяти, хотя это не требуется. Чтобы не делать лишние операции несколько раз, было принято удалить эти повторения, а номера результатов переназначить, чтобы семантика кода осталась той же.

В архитектуре системы команд выделен регистр специального назначения zero, который всегда содержит в себе 0, что позволяет убрать загрузки 0 в качестве отдельной инструкции и добавить ссылки на регистр zero в остальных командах.

После удаления повторяющихся инструкций могут появиться инструкции с одинаковыми операндами и эффектами, они удаляются за избыточностью и номера результатов переназначаются для сохранения семантики кода.

Операция в общем случае необязательная, но после применения предыдущих шагов оптимизации могли появиться инструкции, результат и эффекты которых не важны и/или не используются, они удаляются за ненужностью.

6.6. Генератор кода

6.7. Ассемблер и линковщик

Заключение

В ходе работы поставленная цель: «создать тестовое окружение для верификации вычислительного ядра» была достигнута, все задачи были выполнены. В ходе разработки учитывались проделанные ранее обзор и анализ 2D графических ускорителей, а также разработка велась с упором на разработанную ранее архитектуру графического ускорителя.

Литература

- [1] «Уровень регистровых передач». [Онлайн]. Доступно на: https://ru.wikipedia.org/wiki/%D0%A3%D1%80%D0%BE%D0%B2%D0%B5%D0%BD%D1%8C_%D1%80%D0%B5%D0%B3%D0%B8%D1%81%D1%82%D1%80%D0%BE%D0%B2%D1%8B%D1%85_%D0%BF%D0%B5%D1%80%D0%B5%D0%B4%D0%B0%D1%87
- [2] И.А. Дудкин, Д.М. Старухин, и В.В. Черкасов, «Обзор и анализ 2D графических ускорителей».
- [3] И.А. Дудкин, Д.М. Старухин, и В.В. Черкасов, «Разработка архитектуры аппаратной реализации 2D графического ускорителя».
- [4] *ГОСТ Р 70916-2023. БЛОКИ СЛОЖНО-ФУНКЦИОНАЛЬНЫЕ Термины и определения.*
- [5] «Верификация цифровых схем. Обзор». [Онлайн]. Доступно на: <https://habr.com/ru/articles/481542/>
- [6] «Mesa 3D Documentation». [Онлайн]. Доступно на: <https://docs.mesa3d.org/index.html>
- [7] «LLVMpipe». [Онлайн]. Доступно на: <https://docs.mesa3d.org/drivers/llvmpipe.html>
- [8] «NIR Intermediate Representation (NIR)». [Онлайн]. Доступно на: <https://docs.mesa3d.org/nir/index.html>
- [9] «TGSI». [Онлайн]. Доступно на: <https://docs.mesa3d.org/gallium/tgsi.html>
- [10] *Vulkan Docs*. [Онлайн]. Доступно на: <https://github.com/KhronosGroup/Vulkan-Docs>
- [11] «Three-address code». [Онлайн]. Доступно на: https://en.wikipedia.org/wiki/Three-address_code
- [12] Alfred V. Aho, Monica S. Lam, Ravi Sethi, и Jeffrey D. Ullman, *Compilers: Principles, Techniques, and Tools*. Pearson Education, Inc.

- [13] «Библиотека стандарта RISC-V». [Онлайн]. Доступно на: <https://docs.riscv.org/reference/home/index.html>
- [14] «Язык ассемблера x86». [Онлайн]. Доступно на: https://en.wikipedia.org/wiki/X86_assembly_language